ECE 122: Introduction to Programming for ECE - Spring 2026
Project 3: Chess Engine
Due Date: 05/01/2026
Upload Submissions to Gradescope

Objectives:

This project is designed to reinforce and assess core learning objectives

Students will:

- Implement core game logic using Python fundamentals (variables, functions, lists)
- Apply control flow (if/else statements and loops) to model piece movement and game rules
- Work with 2D lists to represent and update the chess board
- Develop algorithms to generate and validate moves based on chess rules
- Translate user input into structured data that the program can use
- Build and connect multiple functions to create a complete, working system
- Practice debugging and testing by verifying move correctness and game behavior

Project Description:

The goal of this project is to implement the core logic of a **Command Line Interface (CLI) chess game** by working within a pre-built system. Instead of building the entire game from scratch, students will focus on developing the essential components that control how the game behaves, including how pieces move, how moves are validated, and how the game state is updated.

If you are a beginner in chess or wish to revise the rules: Here's a YouTube video you can watch to understand how the game works.

Additionally, you do not need to implement more complex rules like castling, en passant, threefold repetition, fifty-move, insufficient material or a timer system.

In this version:

- The board is displayed in the terminal
- Players enter moves using coordinates (e.g., e2e4)
- The system validates and applies moves
- The game continues until a win condition is reached

Students will NOT build the entire game from scratch. Instead, they will implement key logic components inside an existing system.

Project Files:

main.py	**DO NOT EDIT THIS FILE.** **This file controls the main game loop and user interaction.** ● displays the board ● accepts user commands ● applies moves entered by the player ● allows the engine to make moves ● supports commands such as move, hint, ai, depth, side, load, save, log, undo, and new When running the project, always run this file.
board.py	**Implement core board logic and game state management.** ● Stores the current board configuration ● Applies moves to update the board ● Generates possible moves for the current player ● Determines if the game has ended and who won Students will implement several key functions in this file.
eval.py	**Implements board evaluation logic.** ● assigns values to pieces ● uses piece-square tables for positional bonuses ● computes a score for the current board state Students will implement the evaluation functions in this file.
io_utils.py	**DO NOT EDIT THIS FILE.** **Handles saving and loading positions and move logs.** ● loads a board from a text file ● saves a board to a text file

	● saves and loads move list
movegen.py	**DO NOT EDIT THIS FILE.** **Provides move ordering for search.** ● scores moves heuristically ● sorts legal moves so the search examines stronger moves first
pieces.py	**Implement movement logic for chess pieces.** ● contains the base Piece class ● defines pawn, knight, bishop, rook, queen, and king movement ● provides helper logic for sliding and step-based movement ● defines the Move data structure and square conversion helpers Students will implement movement logic for selected pieces.
search.py	**DO NOT EDIT THIS FILE.** **Implements the engine search algorithm.** ● searches for the best move ● uses board evaluation and ordered legal moves ● handles terminal positions such as checkmate and stalemate
tests.py	**DO NOT EDIT THIS FILE.** **Contains test cases for validating your implementation.** ● checks legal move generation in the start position ● checks making and undoing a move ● checks rejection of illegal moves ● checks save/load correctness ● checks that the engine search returns a move Students are encouraged to use this file for testing but should not rely on it alone.

Work Distribution

To ensure equal contribution, the following function split must be followed:

Student 1:

- try_parse_move()
- apply_move()
- generate_pseudo_legal_moves()
- Piece._step_moves()
- Pawn.pseudo_legal_moves()
- Knight.pseudo_legal_moves()

Student 2:

- generate_legal_moves()
- is_game_over()
- result()
- Piece._slide_moves()
- Rook.pseudo_legal_moves()
- evaluate()

Each student is responsible for implementing and testing the functions assigned to them. Both students are expected to understand the full program structure and how all functions interact when main.py is run.

Function Overview:

Your Chess Engine should support the following way:

Functions in **board.py:**	try_parse_move(), apply_move(), generate_pseudo_legal_moves(), generate_legal_moves(), is_game_over(), result()
Parse Move Input **try_parse_move( )**	Students should focus on converting user input into something the program understands. When a player types a move like e2e4, this function should break it into a starting square and ending square, convert those into board coordinates (row 0 means rank 8, row 7 means rank 1, column 0 means file a, column 7 means file h (rank and file are chess terminologies which are pretty intuitive but don't confuse this 'file' with a python file :).)), and match it with a valid move. If the input is invalid or not a legal move, the function should reject it by raising an error.

	Convert user input into a move object. **Inputs:** ● text: the user's input string representing a move (e.g., "e2e4") **Output:** ● A Move object representing the parsed move ● Raises an error if the input is invalid **How the parameter is used in main.py:** ● The user enters a move as text ● This function converts it into usable board coordinates ● If parsing fails, main.py will reject the move **Rules:** ● must remove spaces and process lowercase input ● must accept 4-character moves and 5-character promotion moves ● must parse source and destination using chess coordinates ● must return the legal matching move if found ● must raise an error for invalid format or illegal moves ● must not modify the board directly Valid Input:

```
> move e2e4
Played e2e4
Engine plays g8f6  (score 41, nodes 1227)

    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 | r | n | b | q | k | b | . | r | 8
  +---+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p | 7
  +---+---+---+---+---+---+---+---+
6 | . | . | . | . | . | n | . | . | 6
  +---+---+---+---+---+---+---+---+
5 | . | . | . | . | . | . | . | . | 5
  +---+---+---+---+---+---+---+---+
4 | . | . | . | . | P | . | . | . | 4
  +---+---+---+---+---+---+---+---+
3 | . | . | . | . | . | . | . | . | 3
  +---+---+---+---+---+---+---+---+
2 | P | P | P | P | . | P | P | P | 2
  +---+---+---+---+---+---+---+---+
1 | R | N | B | Q | K | B | N | R | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
Turn: White
>
```

Invalid Input:

<table>
<tr>
<td></td>
<td>

```
Turn: White
> move z1z2
Error: Invalid square: z1

    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 | r | n | b | q | k | b | n | r | 8
  +---+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p | 7
  +---+---+---+---+---+---+---+---+
6 | . | . | . | . | . | . | . | . | 6
  +---+---+---+---+---+---+---+---+
5 | . | . | . | . | . | . | . | . | 5
  +---+---+---+---+---+---+---+---+
4 | . | . | . | . | . | . | . | . | 4
  +---+---+---+---+---+---+---+---+
3 | . | . | . | . | . | . | . | . | 3
  +---+---+---+---+---+---+---+---+
2 | P | P | P | P | P | P | P | P | 2
  +---+---+---+---+---+---+---+---+
1 | R | N | B | Q | K | B | N | R | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
Turn: White
>
```

File in which the function is present: **board.py**
</td>
</tr>
<tr>
<td>Apply Move
apply_move()</td>
<td>

Students should think about how the board changes after a move. A piece should move from its starting square to its destination square, and the starting square should become empty. If there is an opponent's piece on the destination square, it should be captured. The function must also update whose turn it is and store enough information so the move can be undone later.

Apply a move to the board.

Inputs:

● move: the move object created from user input(eg move e2e4, move g1f3)

Output:

● None

How the parameter is used in main.py:

● After a move is validated, this function updates the board
● The updated board is then used for future turns

Rules:

- must move the piece from source square to destination square
- must store the moved piece, captured piece, and previous turn for undo support
- must clear the source square
- must place the moving piece on the destination square
</td>
</tr>
</table>

- if a pawn reaches the last rank, it must promote
- if no promotion piece is specified, default promotion must be queen
- promotion should be specified like this: promotion input should look like e7e8q, e7e8r, e7e8b, or e7e8n
- must switch the side to move
- must append the move to the history list

Before:

```
CLI Chess Engine
Type 'help' for commands.

    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 | r | n | b | q | k | b | n | r | 8
  +---+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p | 7
  +---+---+---+---+---+---+---+---+
6 | . | . | . | . | . | . | . | . | 6
  +---+---+---+---+---+---+---+---+
5 | . | . | . | . | . | . | . | . | 5
  +---+---+---+---+---+---+---+---+
4 | . | . | . | . | . | . | . | . | 4
  +---+---+---+---+---+---+---+---+
3 | . | . | . | . | . | . | . | . | 3
  +---+---+---+---+---+---+---+---+
2 | P | P | P | P | P | P | P | P | 2
  +---+---+---+---+---+---+---+---+
1 | R | N | B | Q | K | B | N | R | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
Turn: White
>
```

After:

```
> move e2e4
Played e2e4
Engine plays g8f6  (score 41, nodes 1227)

    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 | r | n | b | q | k | b | . | r | 8
  +---+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p | 7
  +---+---+---+---+---+---+---+---+
6 | . | . | . | . | . | n | . | . | 6
  +---+---+---+---+---+---+---+---+
5 | . | . | . | . | . | . | . | . | 5
  +---+---+---+---+---+---+---+---+
4 | . | . | . | . | P | . | . | . | 4
  +---+---+---+---+---+---+---+---+
3 | . | . | . | . | . | . | . | . | 3
  +---+---+---+---+---+---+---+---+
2 | P | P | P | P | . | P | P | P | 2
  +---+---+---+---+---+---+---+---+
1 | R | N | B | Q | K | B | N | R | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
Turn: White
>
```

Showing the board updated after moving the pawn from e2 to e4

File in which the function is present: **board.py**

Generate Pseudo-Legal Moves **generate_pseudo_legal_moves()**	Students should understand that this function gathers all possible moves based only on how pieces move. For example, a rook can move up, down, left, or right; a knight moves in L-shapes; a pawn moves forward and captures diagonally. This function should collect all such moves for the current player without worrying about whether the king is left in check. Generate all possible moves based on piece movement rules. **Inputs:** ● None (uses current board state) **Output:** ● A list of Move objects **How the parameter is used in main.py:** ● called by generate_legal_moves() ● used as the first step in finding all valid moves for the current player **Rules:** ● must loop through every square on the board ● must find only pieces whose color matches self.turn ● must call each piece's pseudo_legal_moves() function ● must combine all such moves into a single list ● must not modify the board File in which the function is present:**board.py**
Generate Legal Moves **generate_legal_moves()**	Students should take the moves from pseudo-legal moves and filter them to ensure they are truly legal. A move is only legal if it does not leave the player's king in check. So for each move, temporarily apply it, check if the king is safe, and only keep the move if it is valid Filter pseudo-legal moves into fully legal moves. **Inputs:** ● None **Output:** ● a list of legal Move objects **How the parameter is used in main.py:**

	● used by move parsing, search, game-over detection, and testing ● determines which moves are actually allowed **Rules:** ● must start from the pseudo-legal move list ● must apply each move temporarily ● must check whether the side that moved is left in check ● must undo the move after testing it ● must keep only the moves that do not leave the mover in check ● must not leave any permanent board changes after filtering File in which the function is present: **board.py**
Check Game Over **is_game_over()**	Students should recognize that the game ends when the current player has no legal moves left. This function should simply check whether the list of legal moves is empty. It does not need to check anything else like missing kings. Determine whether the game has ended. **Inputs:** ● None **Output:** ● A boolean value **How the parameter is used in main.py:** ●Called before and after moves to determine whether the game has ended **Rules:** ● must return True when the current side has no legal moves ● must return False otherwise ● must not modify the board File in which the function is present:**board.py**

Game Result **result()**	Students should determine what kind of ending the game has reached. If there are no legal moves and the player is in check, it is checkmate and the opponent wins. If there are no legal moves but the player is not in check, it is a stalemate (draw). Otherwise, the game is still ongoing. Return the current game result as a string. **Inputs:** ● None **Output:** ● "ongoing" if legal moves still exist ● "<opposite side> wins by checkmate" if the current player has no legal moves and is in check ● "draw by stalemate" if the current player has no legal moves and is not in check **How the parameter is used in main.py:** ● printed when the game is over **Rules:** ● must return "ongoing" if legal moves exist ● must return a checkmate win string if no legal moves exist and the side to move is in check ● must return "draw by stalemate" if no legal moves exist and the side to move is not in check ● must not modify the board Example:

	 File in which the function is present:**board.py**
Functions in pieces.py	Piece._step_moves(), Piece._slide_moves(), Pawn.pseudo_legal_moves(), Knight.pseudo_legal_moves(), Rook.pseudo_legal_moves()
Step-Based Moves **Piece._step_moves()**	Students should focus on pieces that move a fixed distance, like knights and kings. These pieces move to specific squares relative to their current position. The function should check each possible destination and add it if it is inside the board and not occupied by a friendly piece. Generate moves for pieces that move a fixed distance. **Inputs:** ● board: the current board ● r: current row of the piece ● c: current column of the piece ● deltas: a list of relative row/column offsets **Output:** ● A list of Move objects **How the parameters are used:** ● Used by pieces like knight ● Generates moves for fixed offsets

```
> move g2g4
Played g2g4
Engine plays d8h4  (score 99999, nodes 1990)

    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 | r | n | b | . | k | b | n | r | 8
  +---+---+---+---+---+---+---+---+
7 | p | p | p | p | . | p | p | p | 7
  +---+---+---+---+---+---+---+---+
6 | . | . | . | . | p | . | . | . | 6
  +---+---+---+---+---+---+---+---+
5 | . | . | . | . | . | . | . | . | 5
  +---+---+---+---+---+---+---+---+
4 | . | . | . | . | . | . | P | q | 4
  +---+---+---+---+---+---+---+---+
3 | . | . | . | . | . | P | . | . | 3
  +---+---+---+---+---+---+---+---+
2 | P | P | P | P | P | . | . | P | 2
  +---+---+---+---+---+---+---+---+
1 | R | N | B | Q | K | B | N | R | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
Turn: White
Check!
b wins by checkmate
```

	Rules: ● must check each delta exactly once ● must skip targets that are outside the board ● must allow moves to empty squares ● must allow captures of enemy pieces ● must not allow moves onto squares occupied by a friendly piece ● if a capture is possible, the move should record the captured piece ● must not modify the board File in which the function is present:**pieces.py**
Sliding Moves **Piece._slide_moves()**	Students should think about pieces like rooks, bishops, and queens that move continuously in a direction. For example, a rook moves in straight lines until it hits something. This function should keep moving in each direction, adding squares until it either goes out of bounds or hits a piece. If it hits an enemy piece, it can capture and then must stop Generate moves for pieces that move continuously in a direction. **Inputs:** ● board: the current board ● r: current row of the piece ● c: current column of the piece ● dirs: a list of movement directions **Output:** ● A list of Move objects **How the parameters are used:** ● Used by rook (and similar pieces) ● Generates moves along straight lines **Rules:** ● must move repeatedly in each direction until leaving the board or hitting a piece ● must add moves for empty squares ● must add one capture move if the first blocking piece is an enemy ● must stop immediately after hitting any piece

	● must not allow capturing a friendly piece ● if a capture is possible, the move should record the captured piece ● must not modify the board File in which the function is present:**pieces.py**
Pawn Moves **Pawn.pseudo_legal_moves()**	Students should remember that pawns have unique movement rules. A pawn moves forward, but captures diagonally. It can move two squares forward from its starting position if both squares are empty. When it reaches the last rank, it must promote into another piece. This function should carefully handle all these special cases. Generate pawn moves. **Inputs:** ● board: the current board state ● r: row position ● c: column position **Output:** ● A list of Move objects **How the parameters are used:** ● Called when generating moves for pawns **Rules:** ● white pawns move toward smaller row numbers ● black pawns move toward larger row numbers ● a pawn may move one square forward if that square is empty ● a pawn may move two squares forward from its starting row if both squares are empty ● a pawn may capture one square diagonally forward if an enemy piece is present ● if a pawn reaches the promotion row, promotion moves must be generated ● promotion choices must include q, r, b, and n ● capture moves should record the captured piece ● must not modify the board

	Example:
	``
	  a b c d e f g h
	  +---+---+---+---+---+---+---+---+
	8 | r | n | b | q | k | b | n | r | 8
	  +---+---+---+---+---+---+---+---+
	7 | p | p | p | . | p | p | p | p | 7
	  +---+---+---+---+---+---+---+---+
	6 | . | . | . | . | . | . | . | . | 6
	  +---+---+---+---+---+---+---+---+
	5 | . | . | . | p | . | . | . | . | 5
	  +---+---+---+---+---+---+---+---+
	4 | . | . | . | P | . | . | . | . | 4
	  +---+---+---+---+---+---+---+---+
	3 | . | . | N | . | . | . | . | . | 3
	  +---+---+---+---+---+---+---+---+
	2 | P | P | P | . | P | P | P | P | 2
	  +---+---+---+---+---+---+---+---+
	1 | R | . | B | Q | K | B | N | R | 1
	  +---+---+---+---+---+---+---+---+
	  a b c d e f g h
	Turn: Black
	> move e7e5
	Played e7e5
	Engine plays d4e5 (score 229, nodes 1336)
	
	  a b c d e f g h
	  +---+---+---+---+---+---+---+---+
	8 | r | n | b | q | k | b | n | r | 8
	  +---+---+---+---+---+---+---+---+
	7 | p | p | p | . | . | p | p | p | 7
	  +---+---+---+---+---+---+---+---+
	6 | . | . | . | . | . | . | . | . | 6
	  +---+---+---+---+---+---+---+---+
	5 | . | . | . | p | P | . | . | . | 5
	  +---+---+---+---+---+---+---+---+
	4 | . | . | . | . | . | . | . | . | 4
	  +---+---+---+---+---+---+---+---+
	3 | . | . | N | . | . | . | . | . | 3
	  +---+---+---+---+---+---+---+---+
	2 | P | P | P | . | P | P | P | P | 2
	  +---+---+---+---+---+---+---+---+
	1 | R | . | B | Q | K | B | N | R | 1
	  +---+---+---+---+---+---+---+---+
	  a b c d e f g h
	Turn: Black
	``
	
	File in which the function is present: **pieces.py**

Knight Moves **Knight.pseudo_legal_moves()**	Students should focus on the knight's unique L-shaped movement. The knight moves in 8 possible positions and can jump over other pieces. This function should generate those positions and include them if they are within bounds and not occupied by a friendly piece. Generate knight moves. **Inputs:**

<table>
<tr>
<td></td>
<td>
● board: the current board state
● r: row position
● c: column position

Output:

● A list of Move objects

How the parameters are used:

● Called when generating moves for knights

Rules:

<ul>
<li>must generate all 8 possible knight offsets</li>
<li>must use step-based movement</li>
<li>must allow jumping over pieces</li>
<li>must allow moves to empty squares</li>
<li>must allow captures of enemy pieces</li>
<li>must not move onto a friendly piece</li>
<li>must not modify the board</li>
</ul>
Example:

File in which the function is present: pieces.py
</td>
</tr>
<tr>
<td>Rook Moves
Rook.pseudo_legal_moves()</td>
<td>
Students should clearly understand that a rook moves only in straight lines: up, down, left, and right. This function should use sliding movement logic to keep moving in those directions until it is blocked. It should not move diagonally and should not pass through pieces.

Generate rook moves.

Inputs:

● board: the current board state
● r: row position
</td>
</tr>
</table>

	● c: column position
	
	Output:
	
	● A list of Move objects
	
	How the parameters are used:
	
	● Called when generating moves for rooks
	
	Rules:
	
	● must generate moves vertically and horizontally
	● must use sliding movement
	● must continue until blocked
	● must allow captures of enemy pieces
	● must not move through pieces
	● must not capture friendly pieces
	● must not modify the board
	
	Example:
	`> move a1a3`
	`Played a1a3`
	`Engine plays e7e5  (score 408, nodes 1630)`
	
	`  a  b  c  d  e  f  g  h`
	`+---+---+---+---+---+---+---+---+`
	`8 | r | n | . | q | k | b | n | r | 8`
	`+---+---+---+---+---+---+---+---+`
	`7 | p | p | p | . | . | p | p | p | 7`
	`+---+---+---+---+---+---+---+---+`
	`6 | . | . | . | . | . | . | . | . | 6`
	`+---+---+---+---+---+---+---+---+`
	`5 | . | . | . | p | p | . | . | . | 5`
	`+---+---+---+---+---+---+---+---+`
	`4 | P | . | . | . | . | . | b | . | 4`
	`+---+---+---+---+---+---+---+---+`
	`3 | R | . | . | . | . | N | . | . | 3`
	`+---+---+---+---+---+---+---+---+`
	`2 | . | P | P | P | P | P | P | P | 2`
	`+---+---+---+---+---+---+---+---+`
	`1 | . | N | B | Q | K | B | . | R | 1`
	`+---+---+---+---+---+---+---+---+`
	`  a  b  c  d  e  f  g  h`
	`Turn: White`
	
	File in which the function is present: **pieces.py**

Functions in eval.py	evaluate()
Evaluate Board **evaluate()**	Students should understand that this function assigns a score to the board to represent how good the position is. It should calculate a score from White's perspective (using piece values and position), then adjust the sign depending on whose turn it is. The board should not be changed during this process. Return a numeric evaluation of the current position from the perspective of the side to move **Inputs:** ● board: the current board state **Output:** ● An integer score **How the parameter is used in main.py:** ● called by the search algorithm when scoring positions **Rules:** ● must first compute a white-centered score using white_score() ● if board.turn == "w", return that score directly ● if board.turn == "b", return the negated score ● must not leave the board changed after evaluation File in which the function is present: **eval.py**

The logical flow of these files is something like this:

- main.py gets text input
- try_parse_move() validates that text
- generate_legal_moves() filters candidate moves
- piece classes create pseudo-legal moves based on their specific moveset
- apply_move() updates the board
- evaluate() and search.py end up using that resulting board

Grading Criteria (100 Points Total)

Understanding Test Cases

Test cases are prewritten scripts designed to check if your code works correctly. Instead of manually testing every function, these test cases will run specific scenarios automatically and print whether your implementation is correct.

While each test case must independently pass, they collectively should function as a unit to earn full credit.

Both partners must work on the test cases together.

Test 1	Move Parsing + Move Application **Functions tested**: try_parse_move(), apply_move() **What it checks:** ● valid move text is accepted ● illegal move text is rejected ● the moved piece appears on the destination square ● the source square becomes empty ● turn changes after a move ● undo support remains correct	20 points
Test 2	Pseudo-Legal Piece Movement	25points

	Functions tested: Piece._step_moves(), Piece._slide_moves(), Pawn.pseudo_legal_moves(), Knight.pseudo_legal_moves(), Rook.pseudo_legal_moves() **What it checks:** ● step-based moves are generated correctly ● sliding moves stop correctly when blocked ● pawn forward moves, double moves, captures, and promotions work correctly ● knight L-shaped moves are generated correctly ● rook horizontal and vertical moves are generated correctly ● friendly pieces are not captured ● out-of-bounds moves are not generated	
Test 3	Board Move Collection **Functions tested:** generate_pseudo_legal_moves() **What it checks:** ● only pieces of the side to move generate moves ● all pseudo-legal moves are collected into one list ● the returned objects are moves ● the board is not modified during generation	10 points
Test 4	Legal Move Filtering + Game End Detection	20 points

	Functions tested: generate_legal_moves(), is_game_over(), result() **What it checks:** ● legal move generation removes moves that leave the mover in check ● positions with no legal moves are detected correctly ● checkmate returns the correct result string ● stalemate returns the correct result string ● ongoing positions return "ongoing"	
Test 5	Board Evaluation **Functions tested:** evaluate() **What it checks:** ● board positions receive integer scores ● favorable positions for the side to move have better evaluations ● evaluation uses board state without permanently changing it	10 points
Criteria 1.0	Structure & Comments **Explaining Code** Code should be properly commented and formatted for readability. Replace “pass” with your code in the functions.	15 points

Submission Guidelines:

This is a two-person project. Add both names, emails, and SPIRE IDs to this spreadsheet:

If you do not have a partner, add your name and email and we will assign one.
Submission Guidelines:
Submit your

board.py
pieces.py
eval.py

on Gradescope.
Make sure all required functions are correctly implemented. Only use programming concepts discussed in class .
Using AI or copying other students' code is not allowed and will result in grade deductions and
possible disciplinary action.

This project is designed to be implemented incrementally. You are strongly encouraged to run and test each function as you build it.
If you have any questions, reach out to the course TAs.
Happy coding!